

Distributed Systems

Lecture22: Replica Management and Consistency

Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Replica Placement

- **Essence**

  Figure out what the best K places are out of N possible locations.

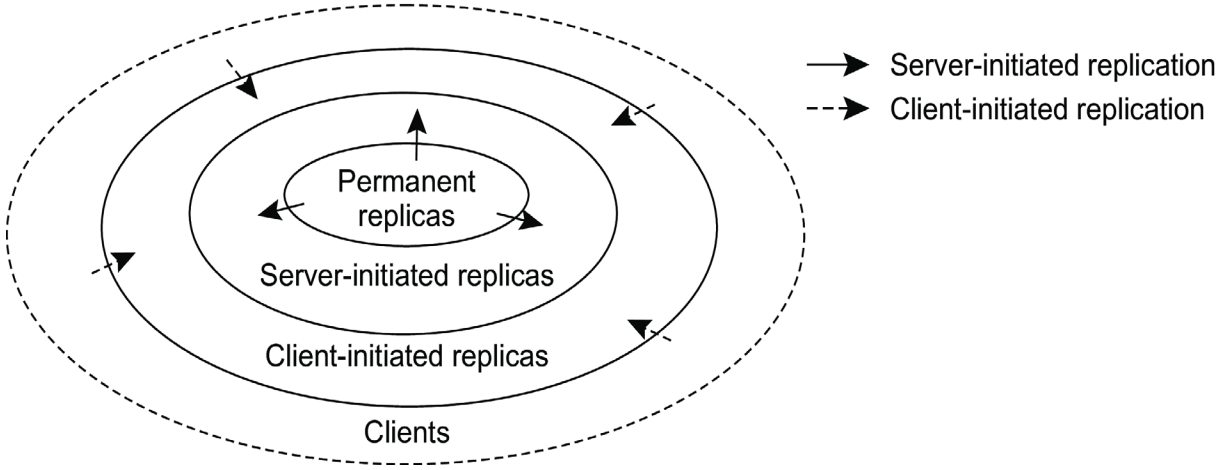
  - Select best location out of N −K for which the average distance to clients is minimal. Then choose the next best server. (Note: The first chosen location minimizes the average distance to all clients.) Computationally expensive.
  - Select the K -th largest autonomous system and place a server at the best-connected host. Computationally expensive.
  - Position nodes in a d -dimensional geometric space, where distance reflects latency. Identify the K regions with highest density and place a server in every one. Computationally cheap.

Content Replication

- Distinguish different processes
- A process is capable of hosting a replica of an object or data:
  - Permanent replicas: Process/machine always having a replica
  - Server-initiated replica: Process that can dynamically host a replica on request of another server in the data store
  - Client-initiated replica: Process that can dynamically host a replica on request of a client (client cache)

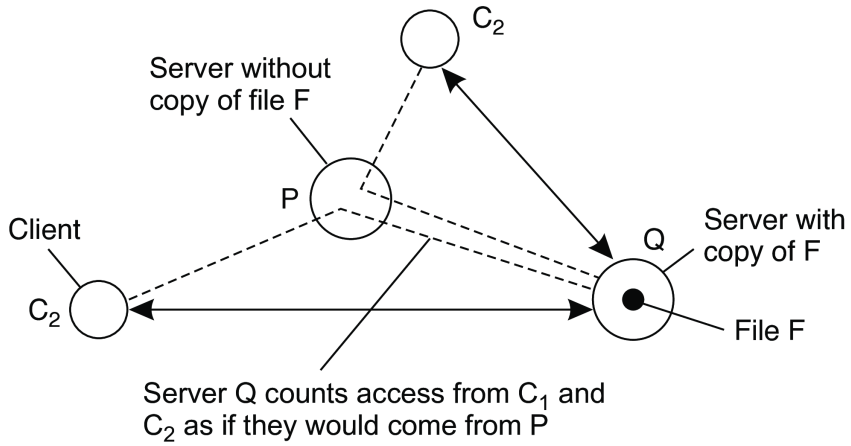
Content Replication

- The logical organization of different kinds of copies of a data store into three concentric rings



Server-initiated Replicas

- Counting access requests from different clients



- Keep track of access counts per file, aggregated by considering server closest to requesting clients
- Number of accesses drops below threshold D ⇒ drop file
- Number of accesses exceeds threshold R ⇒ replicate file
- Number of access between D and R ⇒ migrate file

Content Distribution

- Consider only a client-server combination
  - Propagate only notification/invalidation of update (often used for caches)
  - Transfer data from one copy to another (distributed databases): passive replication
  - Propagate the update operation to other copies: active replication

- **Note**

  No single approach is the best, but depends highly on available bandwidth and read-to-write ratio at replicas.

Content Distribution: Client/Server System

- A comparison between push-based and pull-based protocols in the case of multiple-client, single-server systems
  - Pushing updates: server-initiated approach, in which update is propagated regardless whether target asked for it.
  - Pulling updates: client-initiated approach, in which client requests to be updated.

Issue	Push-based	Pull-based
State at server	List of client replicas and caches	None
Messages sent	Update (and possibly fetch update later)	Poll and update
Response time at client	Immediate (or fetch-update time)	Fetch-update time

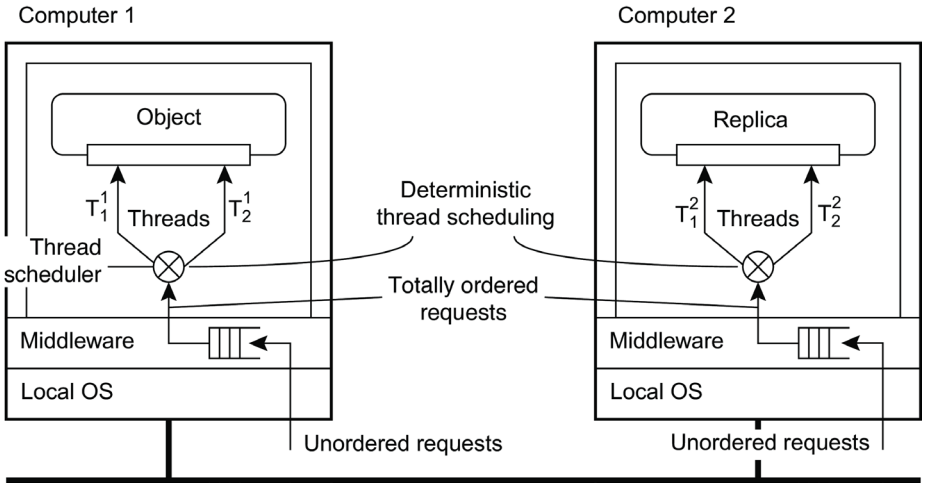
Content Distribution

- **Observation**
- We can dynamically switch between pulling and pushing using leases: A contract in which the server promises to push updates to the client until the lease expires.
- Make lease expiration time adaptive
  - Age-based leases: An object that hasn't changed for a long time, will not change in the near future, so provide a long-lasting lease
  - Renewal-frequency based leases: The more often a client requests a specific object, the longer the expiration time for that client (for that object) will be
  - State-based leases: The more loaded a server is, the shorter the expiration times become
- **Question**

  Why are we doing all this?

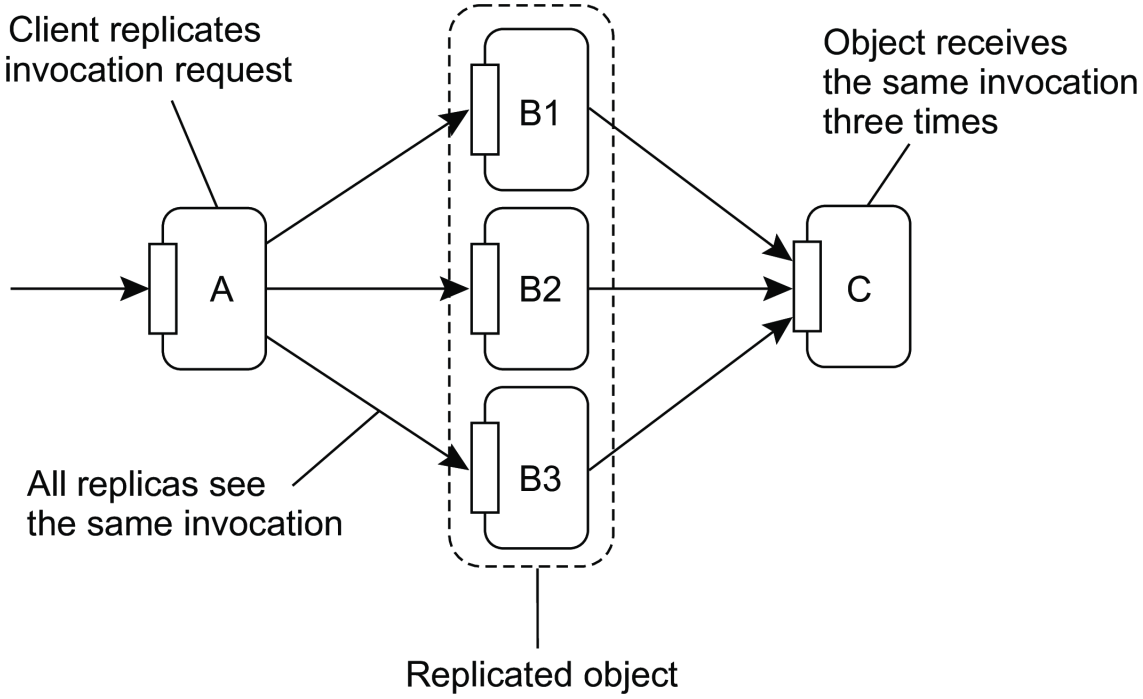
Managing Replicated Objects

- Prevent concurrent execution of multiple invocations on the same object: access to the internal data of an object has to be serialized. Using local locking mechanisms are sufficient.
- Ensure that all changes to the replicated state of the object are the same:
  - No two independent method invocations take place on different replicas at the same time: we need deterministic thread scheduling.

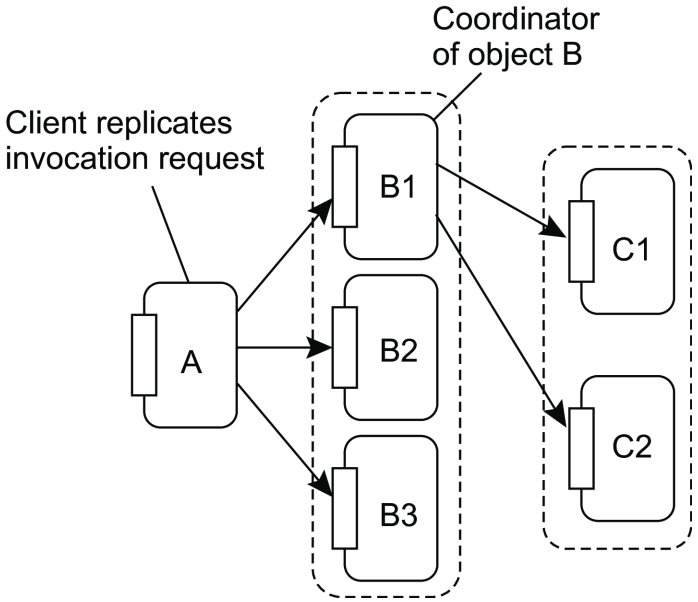


Replicated-object Invocations

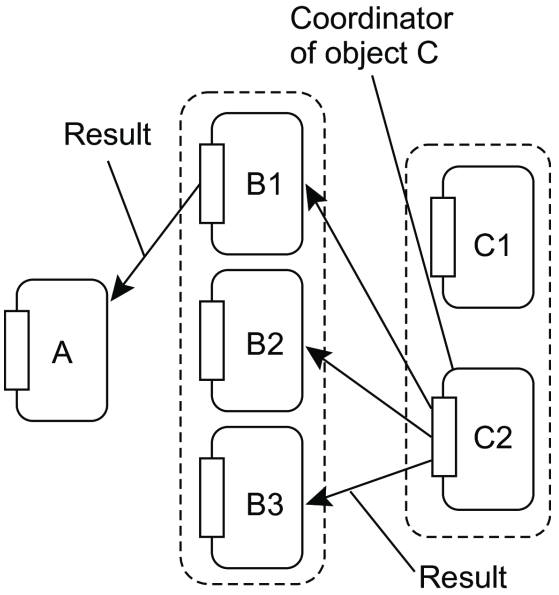
- Problem when invocating a replicated object



Replicated-object Invocations



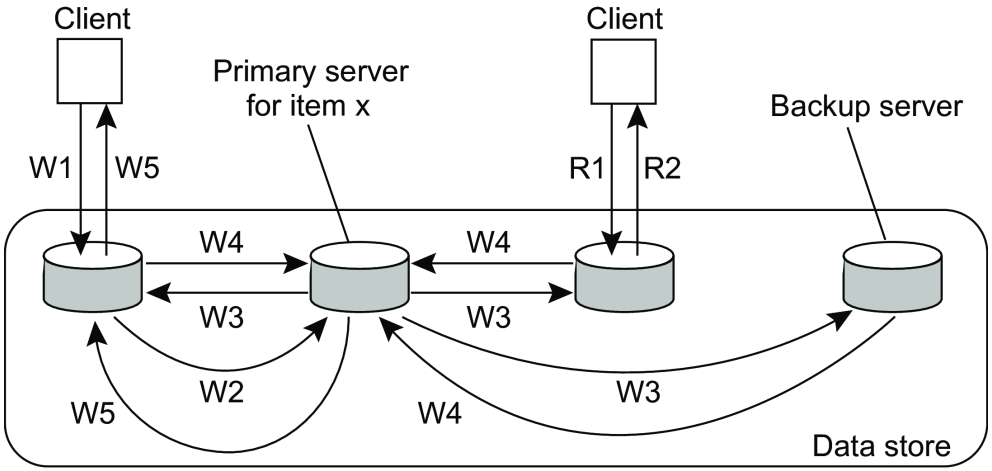
Forwarding a request



Returning the reply

Primary-based Protocols

- Primary-backup protocol



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

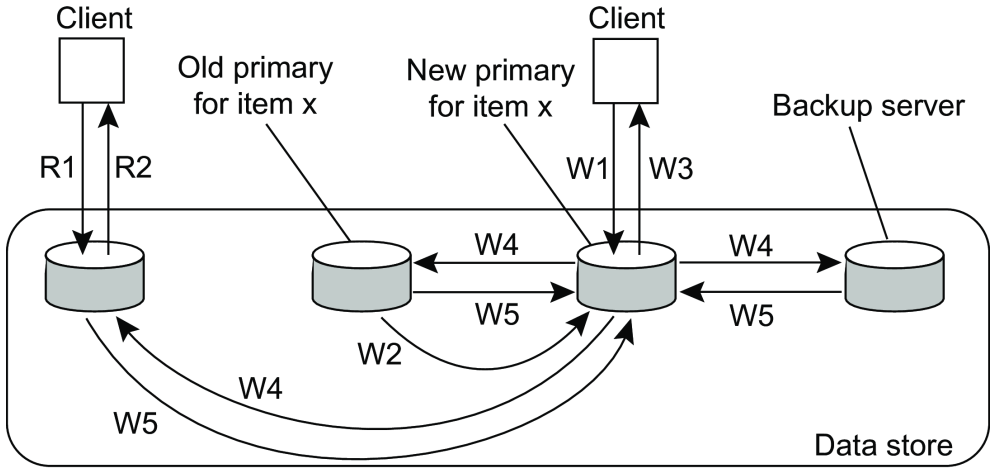
R1. Read request
R2. Response to read

- **Example primary-backup protocol**

  Traditionally applied in distributed databases and file systems that require a high degree of fault tolerance. Replicas are often placed on the same LAN.

Primary-based Protocols

- Primary-backup protocol with local writes



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

- **Example primary-backup protocol with local writes**

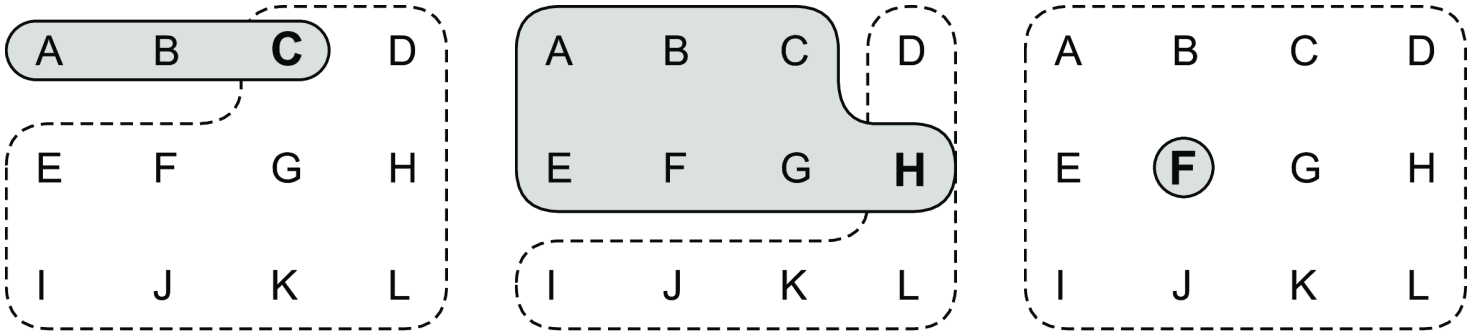
  Mobile computing in disconnected mode (ship all relevant files to user before disconnecting, and update later on).

Replicated-write Protocols

- **Quorum-based protocols**

  Assume N replicas. Ensure that each operation is carried out in such a way that a majority vote is established: distinguish read quorum N_R and write quorum N_W . Ensure:

  - $N_R + N_W > N$ (prevent read-write conflicts)
  - $N_W > N/2$ (prevent write-write conflicts)



Continuous Consistency: Numerical Errors

- **Principal operation**
  - Every server S_i has a log, denoted as L_i .
  - Consider a data item x and let $val(W)$ denote the numerical change in its value after a write operation W . Assume that

$$\forall W : val(W) > 0$$

  - W is initially forwarded to one of the N replicas, denoted as $origin(W)$. $TW\ [i,j]$ are the writes executed by server S_i that originated from S_j:

$$\mathrm{TW}\,[\mathrm{i},\mathrm{j}] = \sum\{\mathrm{val}(\mathrm{W})|\mathrm{origin}(\mathrm{W}) = \mathrm{S_j}\ \&\ \mathrm{W} \in \mathrm{L_i}\}$$

Continuous Consistency: Numerical Errors

- **Problem**
- We need to ensure that $v(t) - v_i < \delta_i$ for every server S_i .
- **Approach**
- Let every server S_k maintain a view $TW_k[i,j]$ of what it believes is the value of $TW[i,j]$. This information can be gossiped when an update is propagated.
- **Note**

$$0 \leq TW_k[i,j] \leq TW[i,j] \leq TW[j,j]$$

Continuous Consistency: Numerical Errors

- **Solution**
- S_k sends operations from its log to S_i when it sees that $TW_k[i,k]$ is getting too far from $TW[k,k]$, in particular, when

$$TW[k,k] - TW_k[i,k] > \frac{\delta_i}{N-1}$$

- **Question**
- To what extent are we being pessimistic here: where does $\delta_i/(N-1)$ come from?
- **Note**
- Staleness can be done analogously, by essentially keeping track of what has been seen last from S_i (see book).

Implementing Client-centric Consistency

- **Keeping it simple**

  Each write operation W is assigned a globally unique identifier by its origin server. For each client, we keep track of two sets of writes:

  - Read set: the (identifiers of the) writes relevant for that client's read operations
  - Write set: the (identifiers of the) client's write operations.

- **Monotonic-read consistency**

  When client C wants to read at server S, C passes its read set. S can pull in any updates before executing the read operation, after which the read set is updated.

- **Monotonic-write consistency**

  When client C wants to write at server S, C passes its write set. S can pull in any updates, executes them in the correct order, and then executes the write operation, after which the write set is updated.

Implementing Client-centric Consistency

- **Read-your-writes consistency**

  When client C wants to read at server S, C passes its write set. S can pull in any updates before executing the read operation, after which the read set is updated.

- **Writes-follows-reads consistency**

  When client C wants to write at server S, C passes its read set. S can pull in any updates, executes them in the correct order, and then executes the write operation, after which the write set is updated.

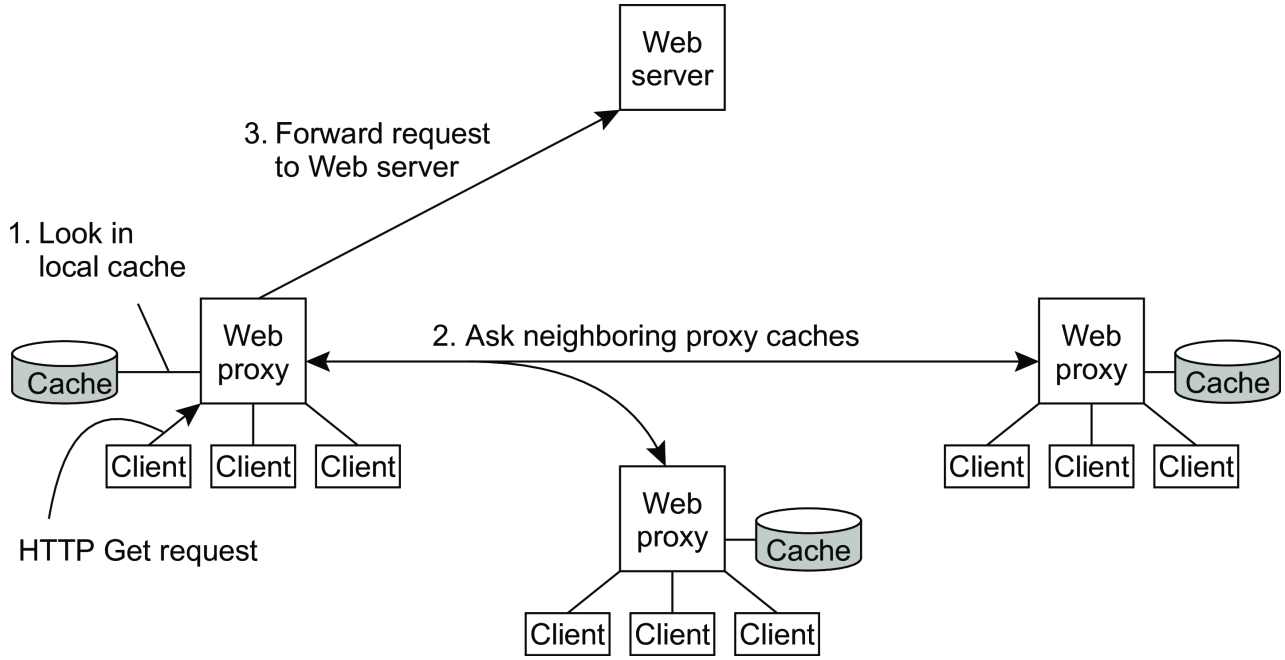
Example: Replication in The Web

- **Client-side caches**
  - In the browser
  - At a client's site, notably through a Web proxy

- **Caches at ISPs**

  Internet Service Providers also place caches to (1) reduce cross-ISP traffic and (2) improve client-side performance. May get nasty when a request needs to pass many ISPs.

Cooperative Caching



Web-cache Consistency

- **How to guarantee freshness?**

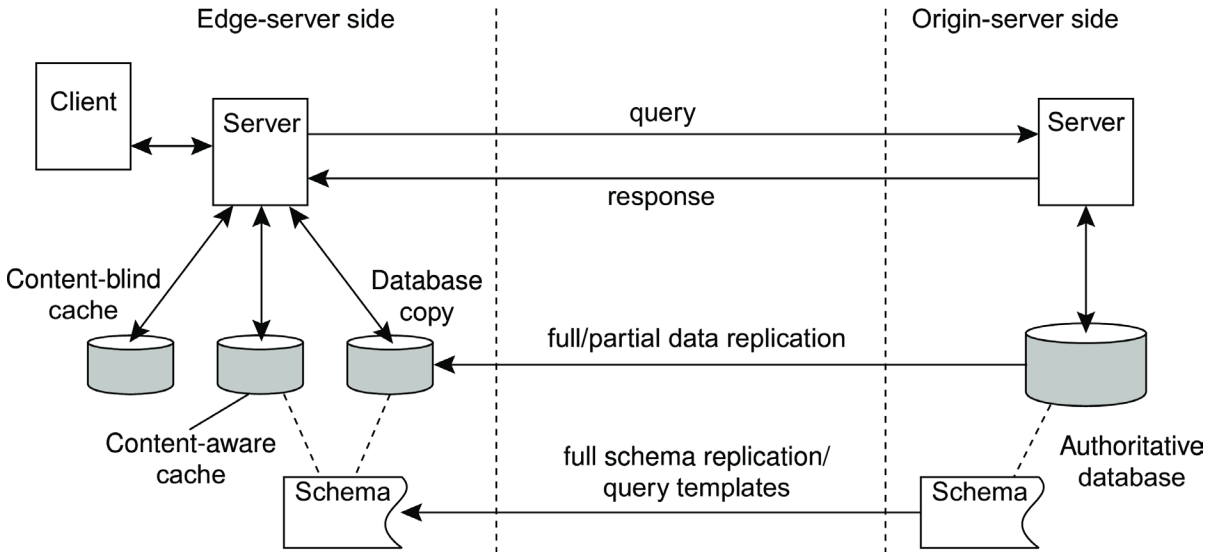
  To prevent that stale information is returned to a client:

  - Option 1: let the cache contact the original server to see if content is still up to date.
  - Option 2: Assign an expiration time T_{expire} that depends on how long ago the document was last modified when it is cached. If $T_{last\ modified}$ is the last modification time of a document (as recorded by its owner), and T_{cached} is the time it was cached, then

$$T_{\text{expire}} = \alpha(T_{\text{cached}} - T_{\text{last modified}}) + T_{\text{cached}}$$

    with $\alpha = 0.2$. Until T_{expire}, the document is considered valid.

Alternatives for Caching and Replication



- **Database copy:** the edge has the same as the origin server
- **Content-aware cache:** check if a (normal query) can be answered with cached data. Requires that the server knows about which data is cached at the edge.
- **Content-blind cache:** store a query, and its result. When the exact same query is issued again, return the result from the cache.